

Lecture 8

Convolutional Neural Networks

Elena Raponi | Neural Computing course



Universiteit
Leiden
The Netherlands

Discover the world at Leiden University

Index

1. Motivation
2. Image Classification
3. Spatial Structure
4. Feature Extraction
5. Padding
6. Convolution
7. Pooling
8. CNN = Feature Learning + Classification
9. Examples



Motivation

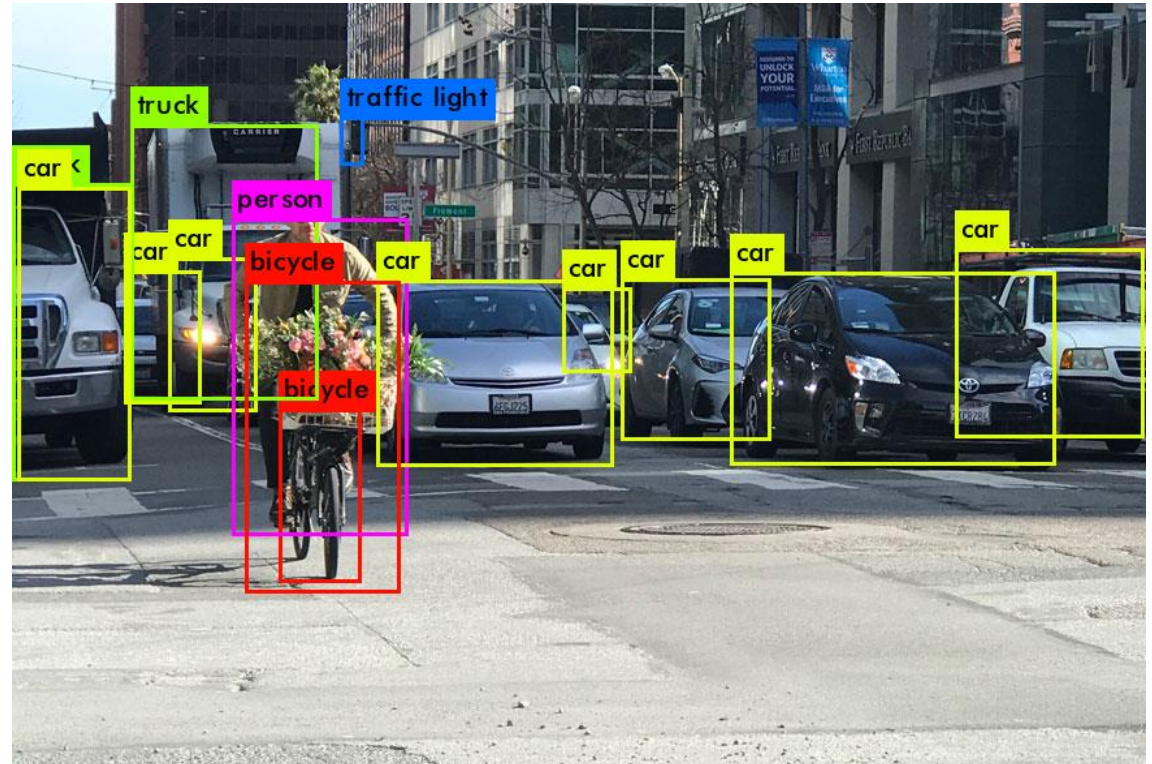
- Sight is one of the most important senses.



- How can we build computers that can achieve the sense of sight and vision?
- In this class we will see how we can use DL and ML to build powerful vision systems that can both **see** and **predict** *what is where*, by only looking at raw visual inputs.

Object detection and prediction

- We need to:
 - build systems that can identify objects
 - understand each of these objects at a much deeper level (predicting the future for example in a traffic scene)
- Our brain can infer all of these very subtle cues
- **How can we build ML/DL algorithms that can achieve that same type and subtle understanding of our world?**



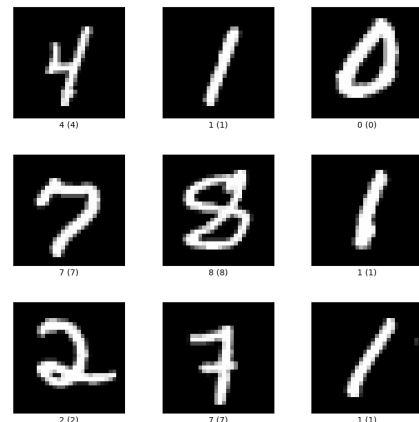
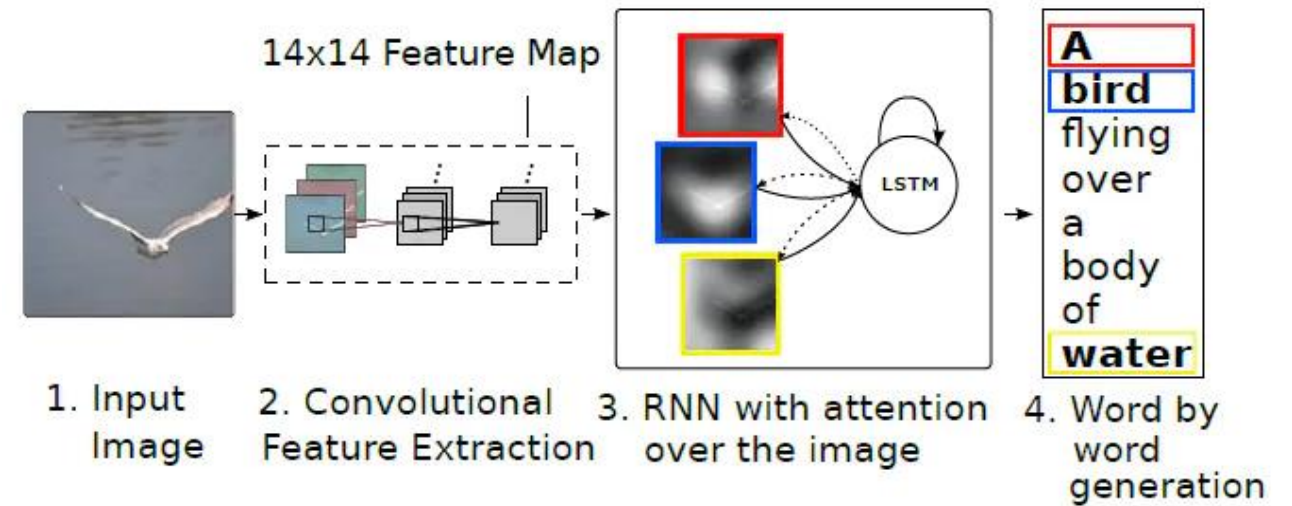
Examples

- Self-driving cars take as input the images and sensing modalities of the vehicle and learn how to predict those actuation commands
- Medicine, biology, healthcare → cancer detection
- Accessibility → helping the visual impaired runners



Several tasks of Computer Vision

- We want to handle digital images
- Computer Vision tasks
 - Image classification
 - Object detection
 - Segmentation
 - Image captioning



Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

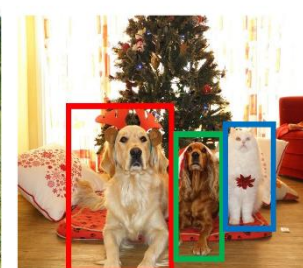
Classification + Localization



CAT

Single Object

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



DOG, DOG, CAT

This image is CC0 public domain

Fundamentals



How does a computer see and process images?



Images are numbers → numerical representation of images



What kind of tasks do we want the machine to do? Regression (real numbers) and classification (labels, categorical values)



In classification, the goal is to output a class probability score.



Robustness is a problem. Features are not robust if we consider all different types of variations within one class. Detecting patterns in the data is not such a simple task.



Power of CNNs: they are able to learn features from data in a hierarchical manner.

Image Classification

- Input image → outputting the probability of classes that best describes the image
- The most natural task that our biological NN handles well
- What the computer sees is different..

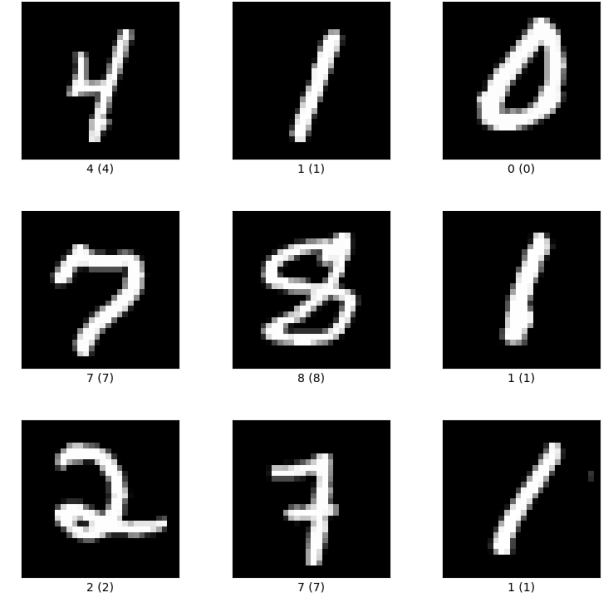


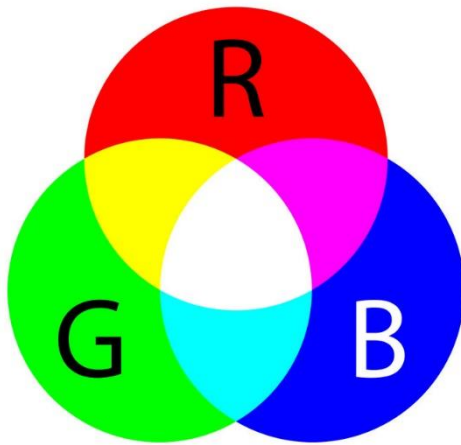
Image Classification

- Gray-scale images (single channel)

```
array([[255, 255, 255, ..., 255, 255, 0],  
       [255, 255, 255, ..., 255, 255, 0],  
       [255, 255, 255, ..., 255, 255, 0],  
       ...,  
       [255, 255, 255, ..., 255, 255, 0],  
       [255, 255, 255, ..., 255, 255, 0],  
       [255, 255, 255, ..., 255, 255, 0]], dtype=uint8)
```



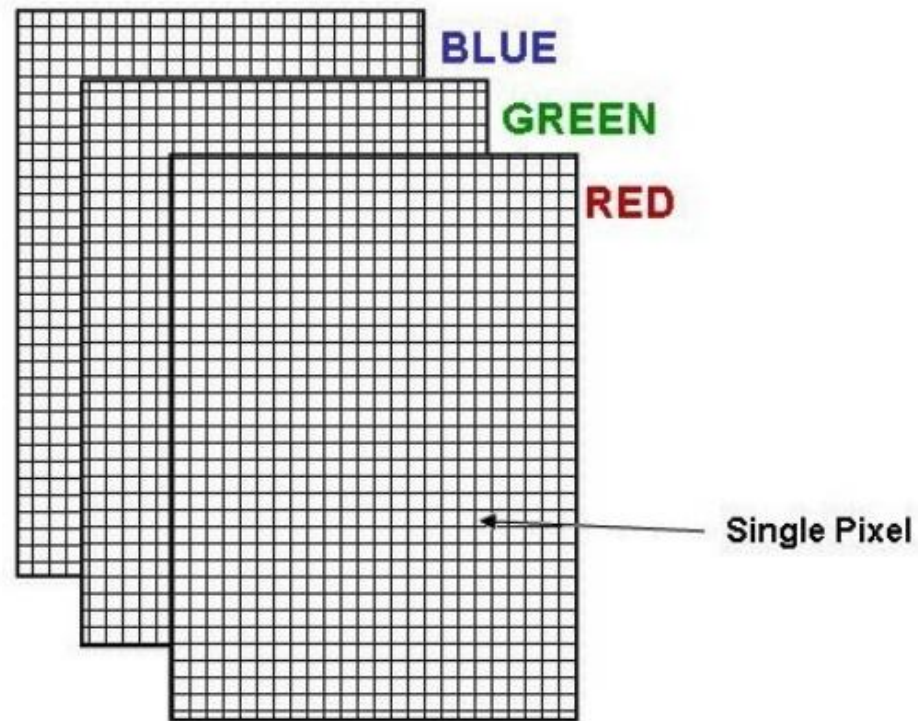
- RGB images (three channels)



```
array([[[ 1, 57, 178, ..., 80, 217, 0],  
        [ 80, 217, 0, ..., 219, 0, 80],  
        [219, 0, 80, ..., 255, 255, 255],  
        ...,  
        [240, 19, 126, ..., 254, 254, 252],  
        [ 3, 123, 233, ..., 208, 249, 186],  
        [208, 247, 190, ..., 239, 20, 128]],  
       [[239, 21, 129, ..., 255, 255, 255],  
        [ 0, 127, 232, ..., 211, 249, 183],  
        [209, 246, 185, ..., 242, 22, 130],  
        ...,  
        [113, 64, 86, ..., 255, 83, 196],  
        [255, 85, 195, ..., 254, 255, 253],  
        [ 79, 184, 250, ..., 60, 17, 46]],  
       [[ 67, 34, 71, ..., 255, 87, 196],  
        [255, 87, 196, ..., 255, 255, 255],  
        [ 80, 183, 250, ..., 63, 8, 41],  
        ...,  
        [110, 163, 21, ..., 199, 30, 147],  
        [205, 32, 148, ..., 25, 143, 203],  
        [ 27, 144, 205, ..., 255, 255, 255]]], dtype=uint8)
```

Image Classification

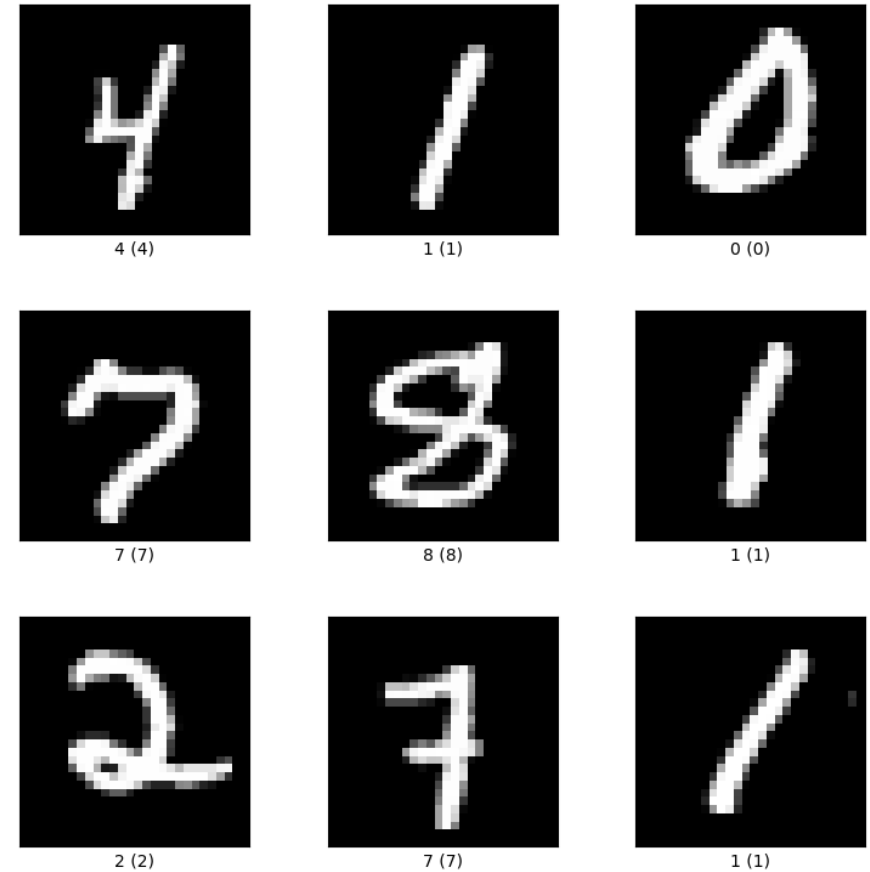
- RGB images (three channels)
 - A “three-dimensional” matrix, formally called a *tensor*



Benchmark Data Sets

MNIST Dataset, <http://yann.lecun.com/exdb/mnist/>

- 60,000 training examples
- 10,000 test examples
- Image size: 28 x 28



Benchmark Data Sets

CIFAR-10 Dataset, <https://www.cs.toronto.edu/~kriz/cifar.html>

- 50,000 training examples
- 10,000 test examples
- Image size: 32 x 32
- 10 classes
- 6000 images per class

airplane



automobile



bird



cat



deer



dog



frog



horse



ship

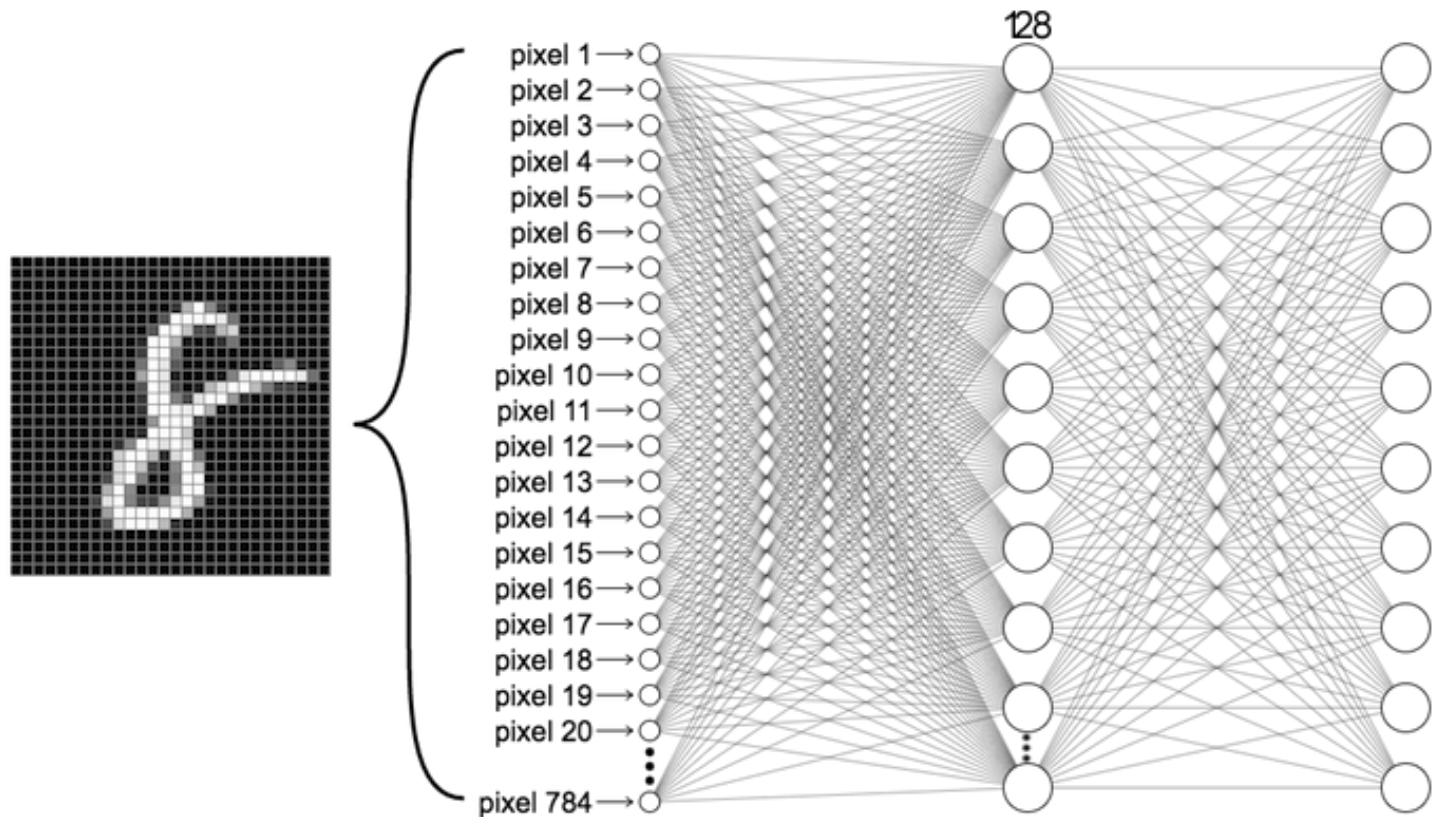


truck



Image Classification: Naïve Approach

- What if we do not introduce convolution to learn visual features?
- We could use a fully connected neural network.
- The image gets flattened.

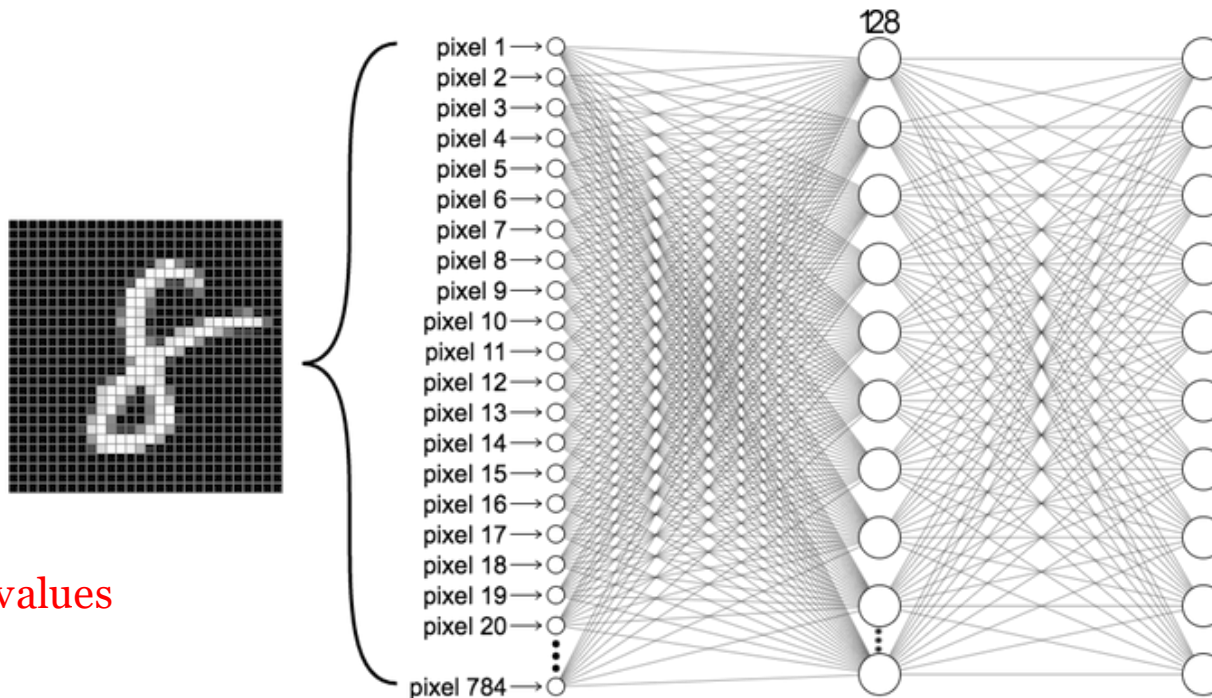


Problem with this naïve approach?

- Why is this approach deprecated?
 - The number of weights ($H \times W$) will explode!
 - We destroy the geometric information of the image!

Input:

- 2D image
- Vector of pixel values

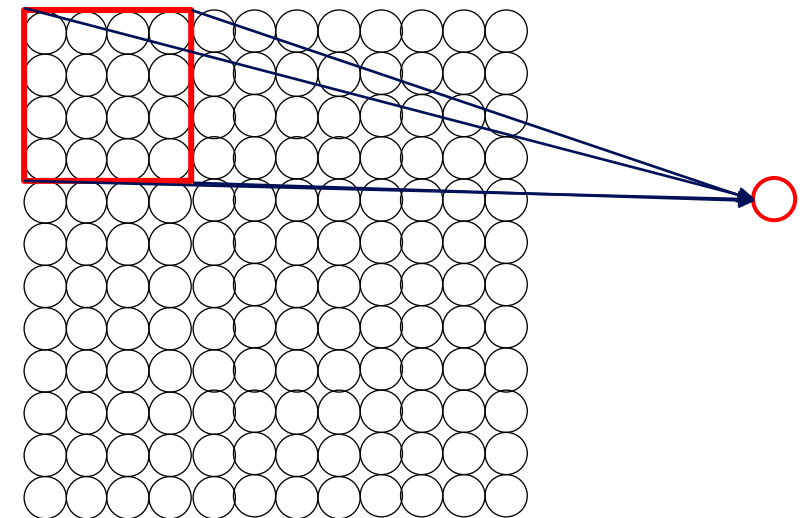


Fully connected:

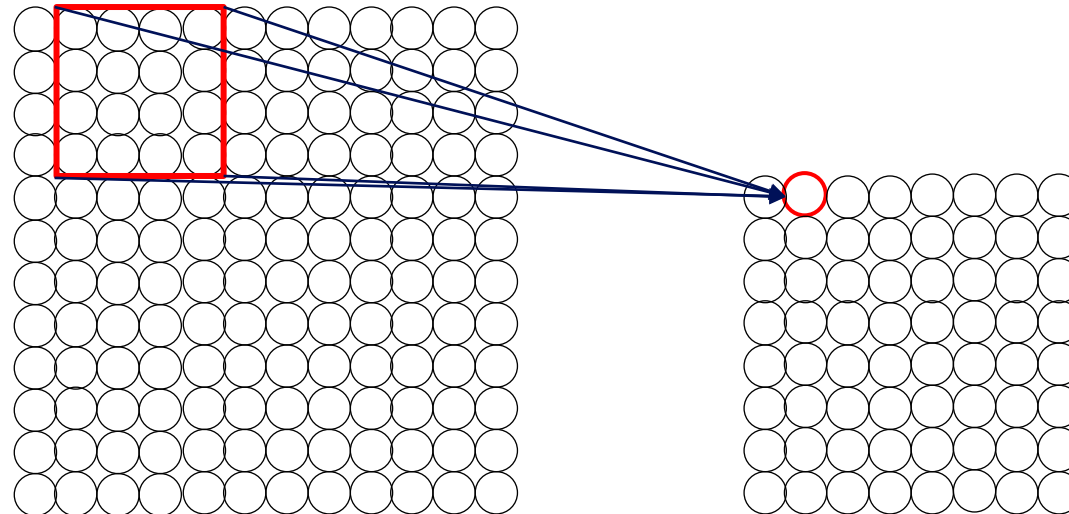
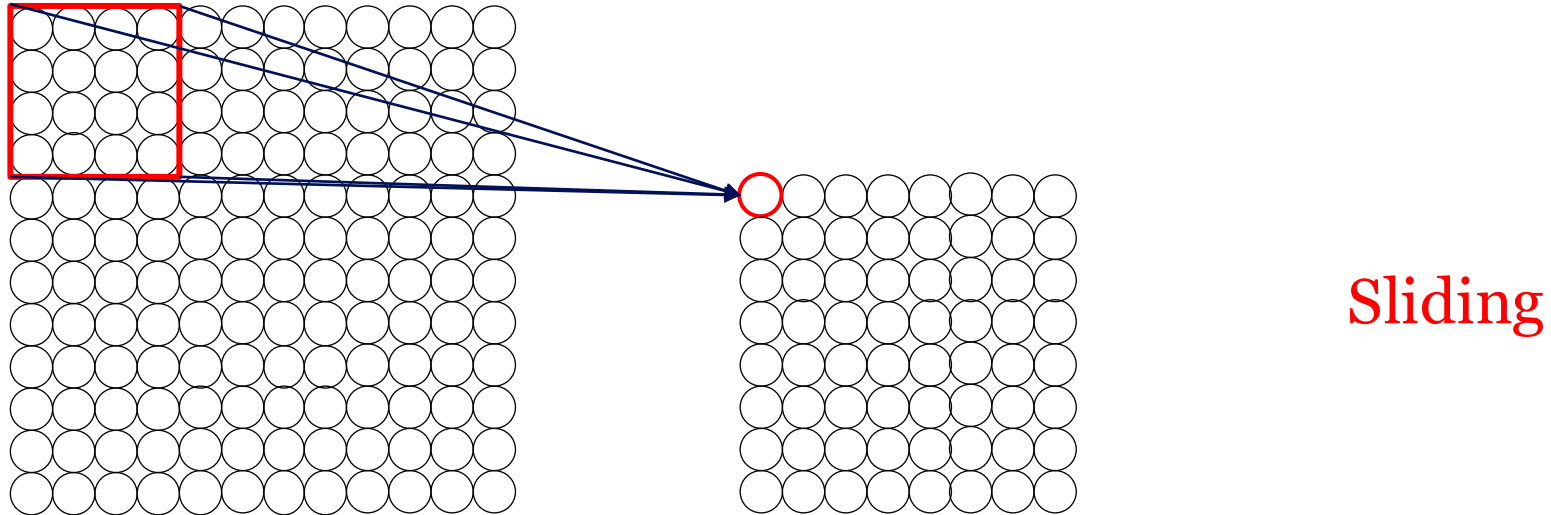
- Connect neuron in hidden layer to all neurons in input layer
- No spatial information
- Many, many parameters!

Using spatial structure

- **Winning Strategy:** Connecting patches of input to neurons in the hidden layer.
- One node only responds to nodes in a patch of the previous layer.
- A single patch corresponds to a single neuron in the subsequent layer.
- We slide the window to define connections.
- How can we weight the patch to detect particular features? Feature extraction with convolution (mathematical operation)
- This “patchy” operation is called **convolution**.



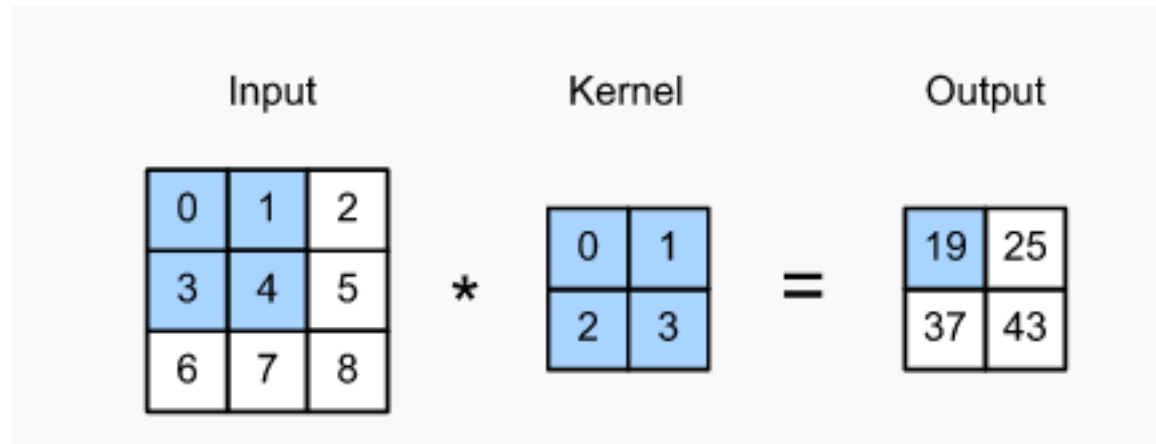
Using spatial structure



Feature Extraction with Convolution

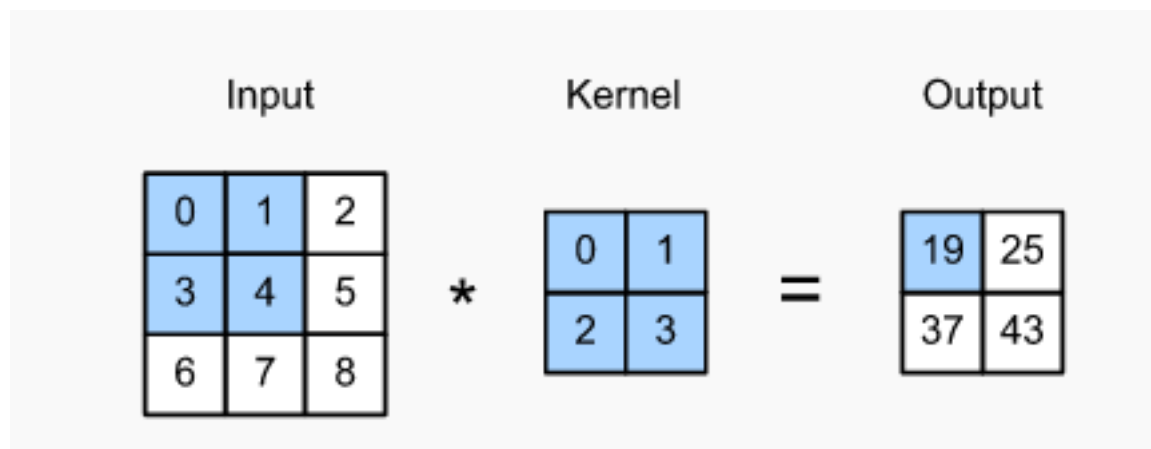
In this example

- **Filter** (a.k.a. **kernel**) of size 2x2 $\rightarrow$ 4 different weights
- The same filter is applied to all 2x2 patches in the input
- Stride: Slide the kernel by 1 pixel for the next patch
- The filter is applied to extract **local features**
- Each filter spatially shares the same parameters

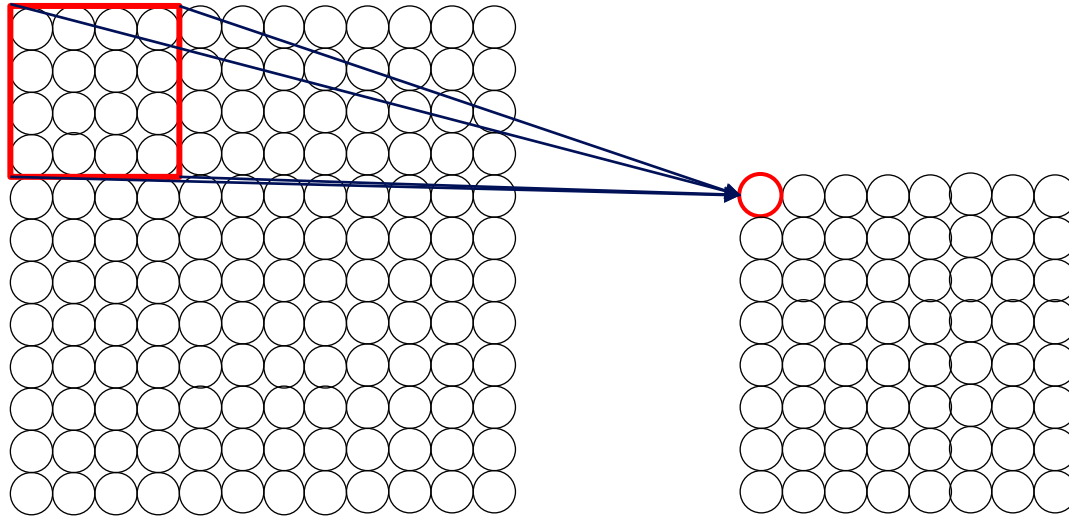


Feature Extraction with Convolution

- Kernel size: (H_k, W_k)
- Image size: $(H_{\text{in}}, W_{\text{in}})$
- Output image/**feature map** size: $(H_{\text{in}} - H_k + 1, W_{\text{in}} - W_k + 1)$



Local Connectivity



4x4 filter = matrix of weights w_{ij}

In the hidden layer:

$$\sum_{i=1}^4 \sum_{j=1}^4 w_{ij} x_{i+p, j+q} + b$$

for each neuron (p,q)

For a neuron in the hidden layer:

- Take inputs from patch
- Compute weighted sum
- Apply bias
- Activate with a non-linear function

Padding

- We might lose information on the edge of images → add padding
- Padding size: (H_p, W_p)
- Output image/**feature map** size: $(H_{\text{in}} - H_k + H_p + 1, W_{\text{in}} - W_k + W_p + 1)$

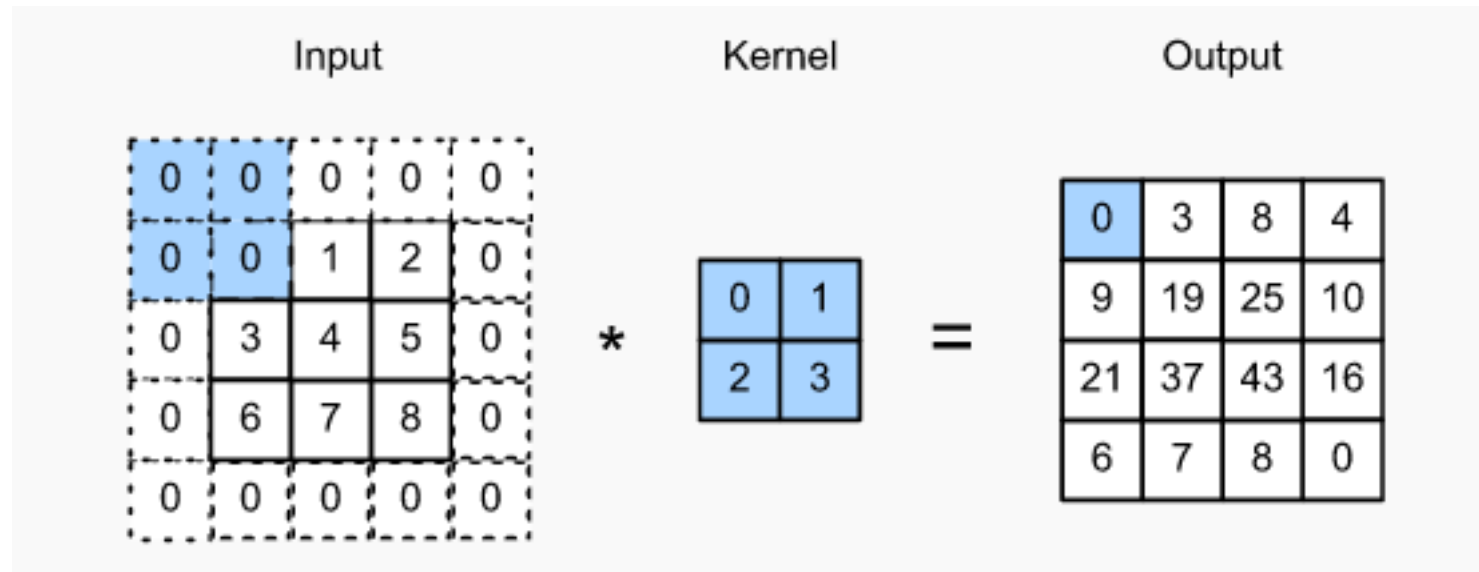


Image Classification: Convolution

Putting everything together:

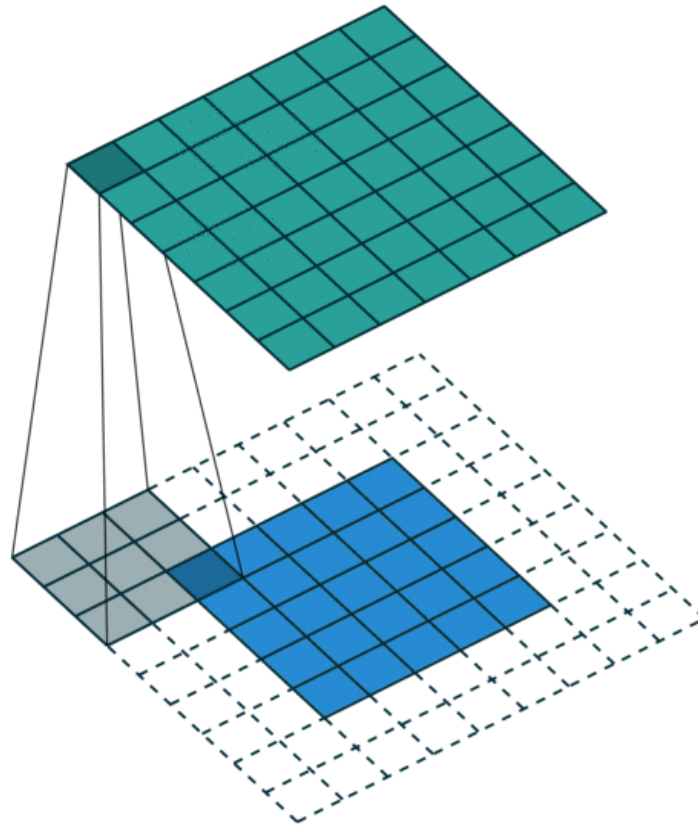
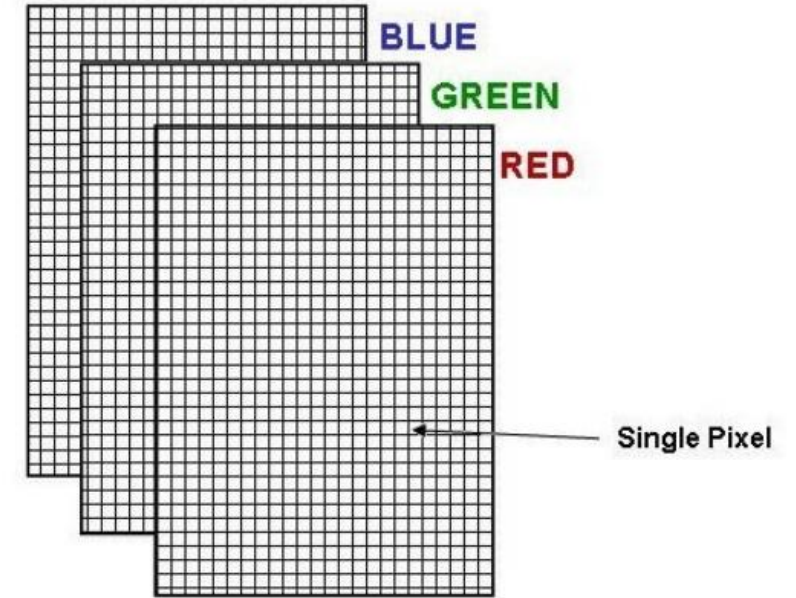
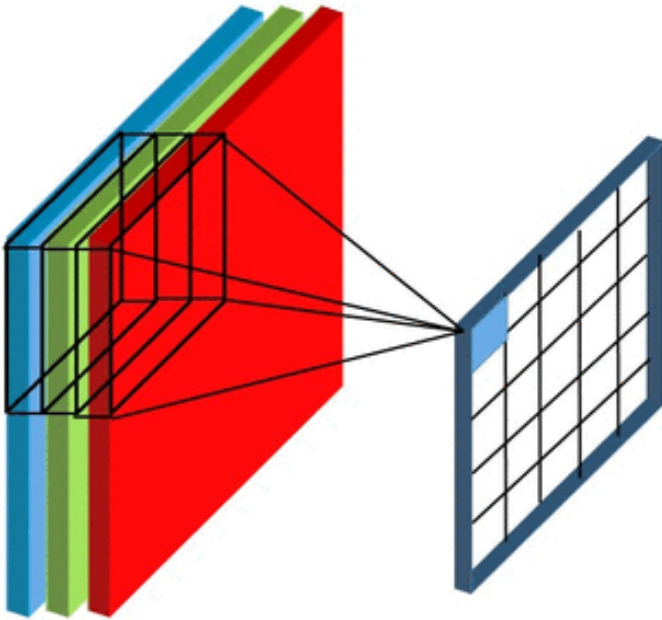


Image Classification: Convolution

How to handle multiple channels?

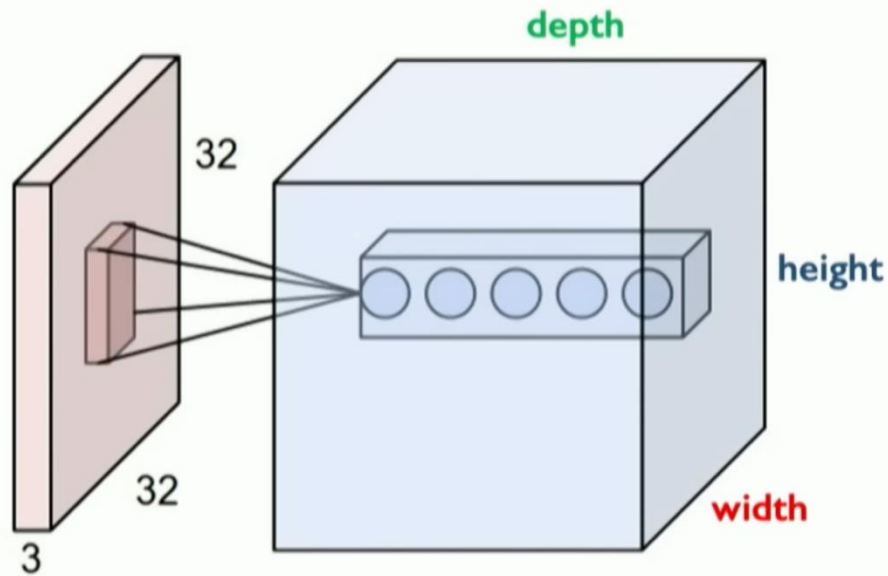
- Individual filter/kernel (of the same size) for each channel
- Add the resulting pixel value from all channels



Convolutional Neural Networks

- Multiple filters must be used to extract multiple features

CNNs: Spatial Arrangement of Output Volume



Layer Dimensions:

$h \times w \times d$

where h and w are spatial dimensions
 d (depth) = number of filters

Stride:

Filter step size

Receptive Field:

Locations in input image that
a node is path connected to

[source: introtodeeplearning.com]

Producing feature maps

- Convolution generates a **feature map**

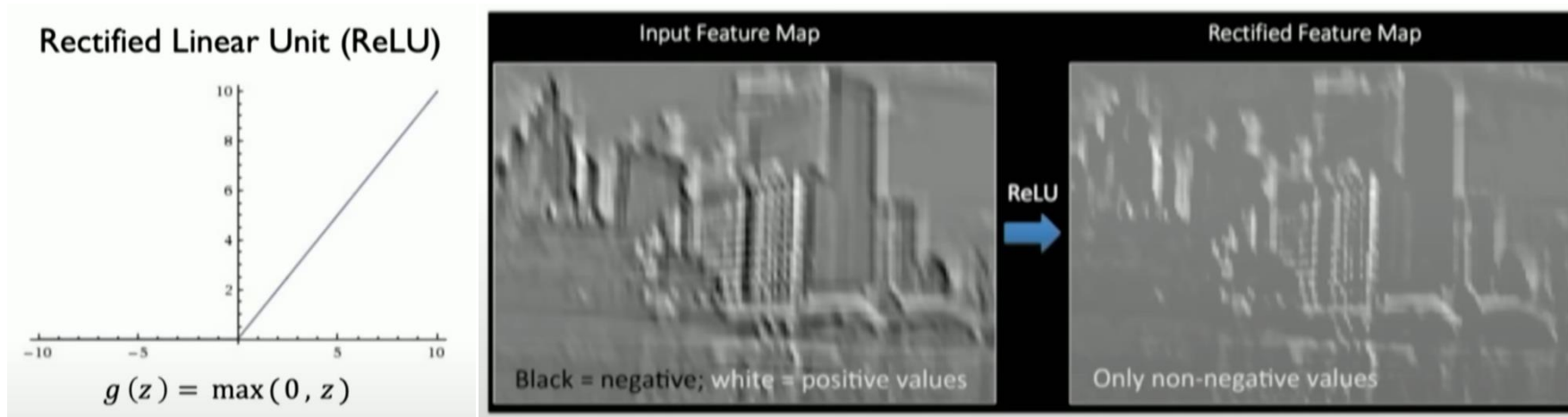
Which feature to use for a task?

How many features to use?

Operation	Kernel	Image result
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	

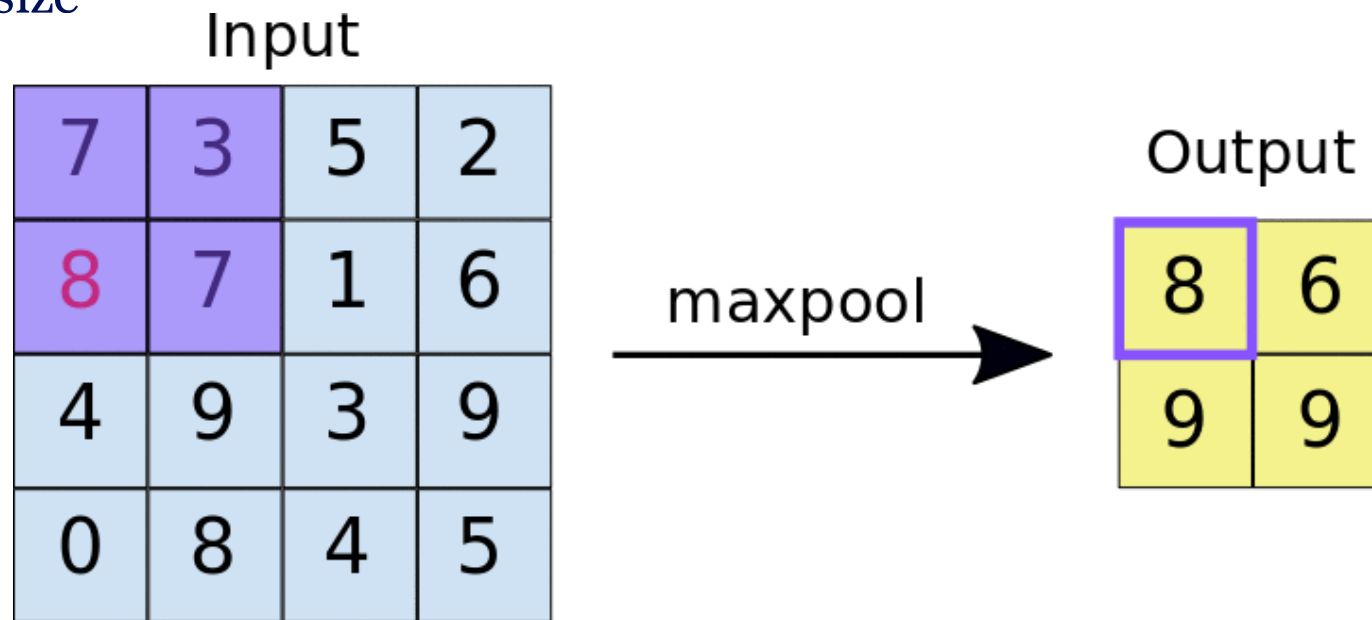
Introducing non-linearities

- We take the resulting feature maps (extracted via the convolutional layers)
- And apply a Rectified Linear Unit (ReLU) function to the output volume of the convolutional layer
- It allows us to deal with non-linear data
- Pixel-by-pixel operation that replaces all negative values (= no detection) by 0 and keeps all positive values the same



Pooling

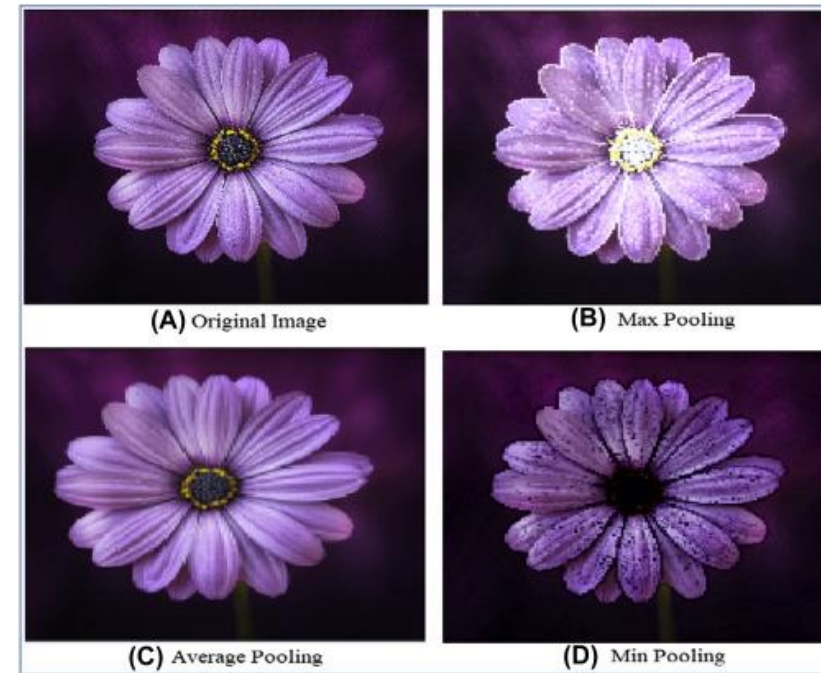
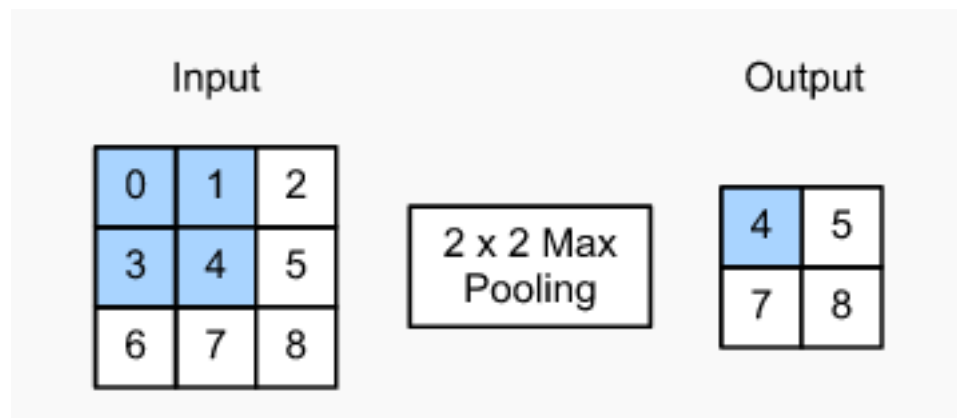
- **Downsample** the feature map
 - Reduce computation cost
 - Extract shape/smooth features
 - Max-pooling/Average-pooling
 - Pooling window size



<https://www.sciencedirect.com/topics/computer-science/convolution-layer>

Pooling

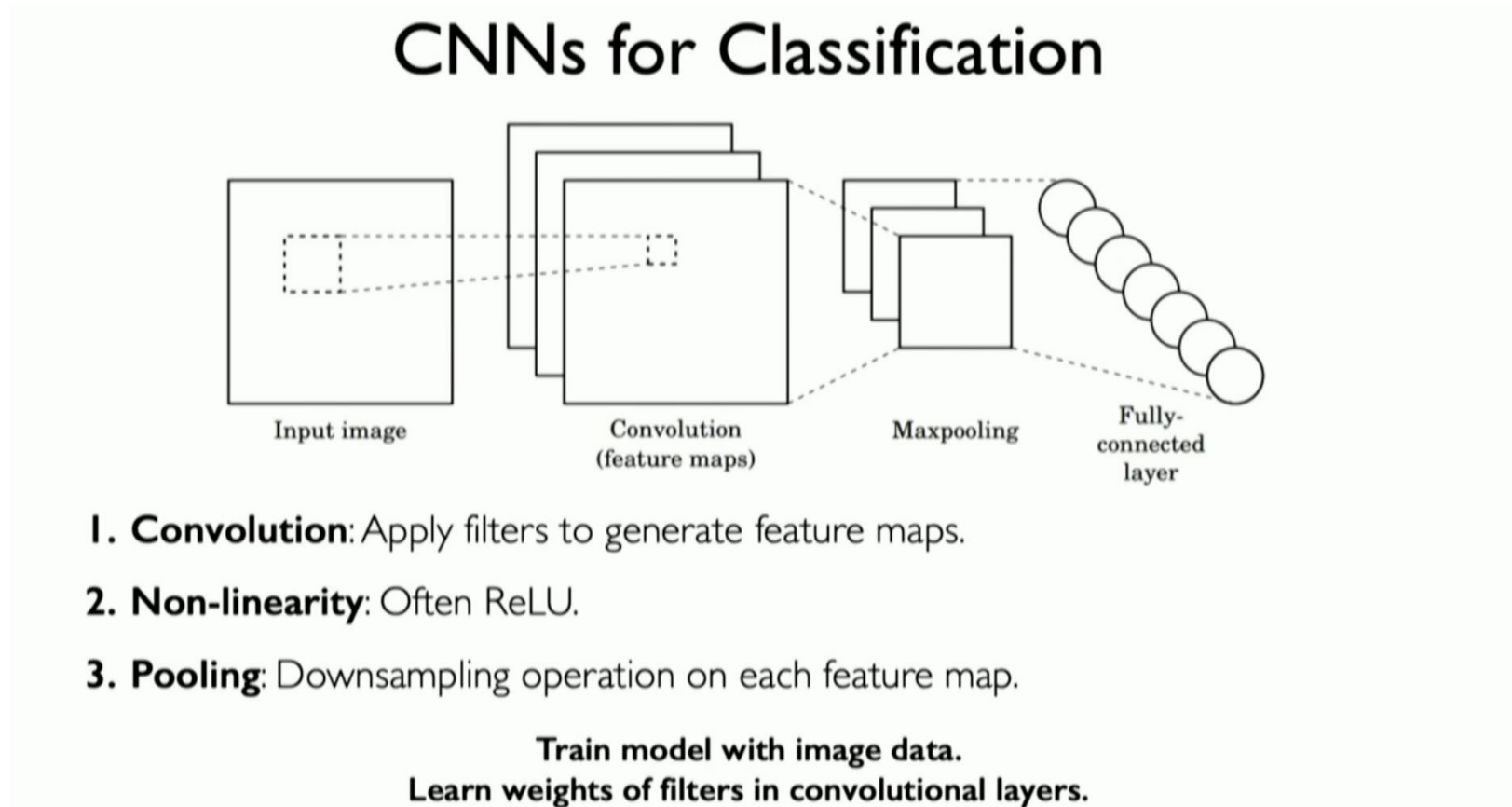
- **Downsample** the feature map
 - Reduce computation cost
 - Extract shape/smooth features
 - Max-pooling/Average-pooling
 - Pooling window size



<https://www.sciencedirect.com/topics/computer-science/convolution-layer>

Convolutional Neural Networks

We need to **learn the filters**, rather than define them.

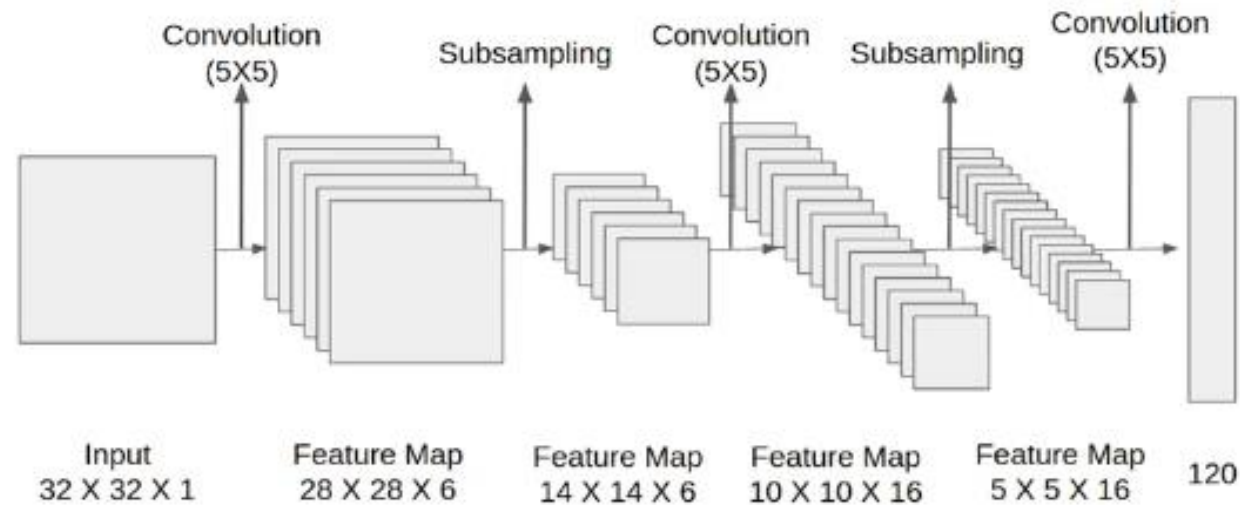


[source: introtodeeplearning.com]

Final step: Fully Connected Layer

“Multilayer Perceptron”

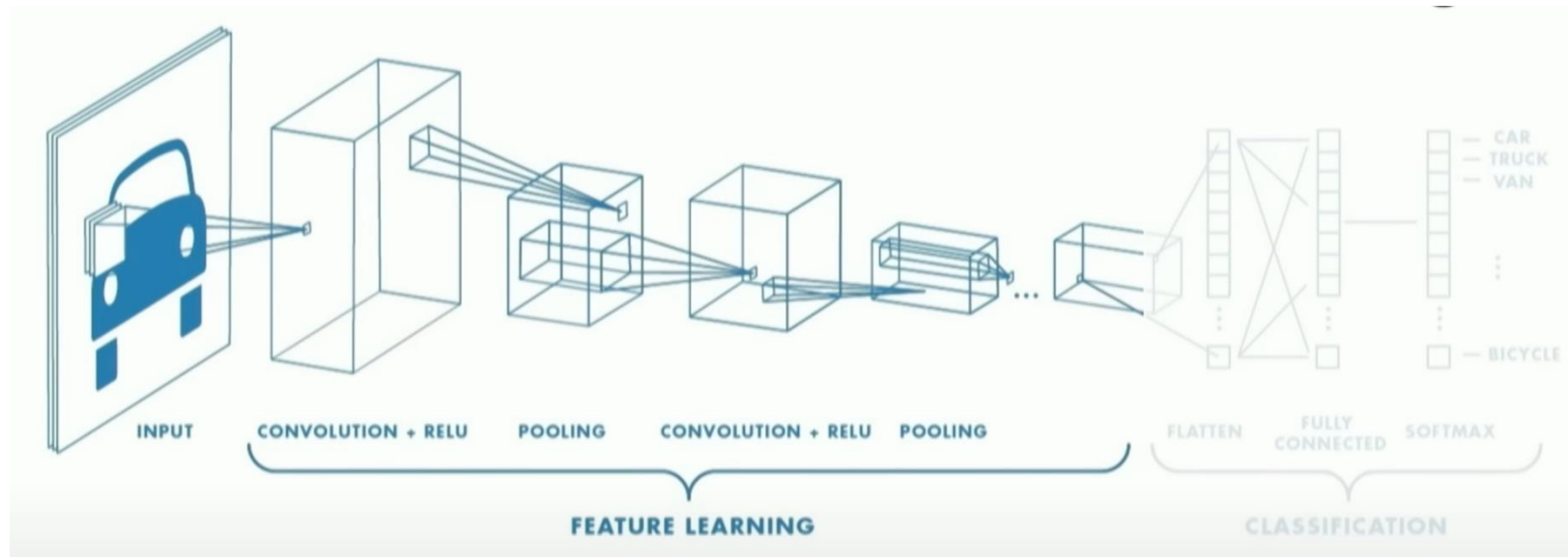
- Input: the final feature maps “**flattened**”



- Multiple layers are needed since the features might not be linearly separable
- Output: classification probabilities (via softmax)

Summary – CNN: Feature Learning

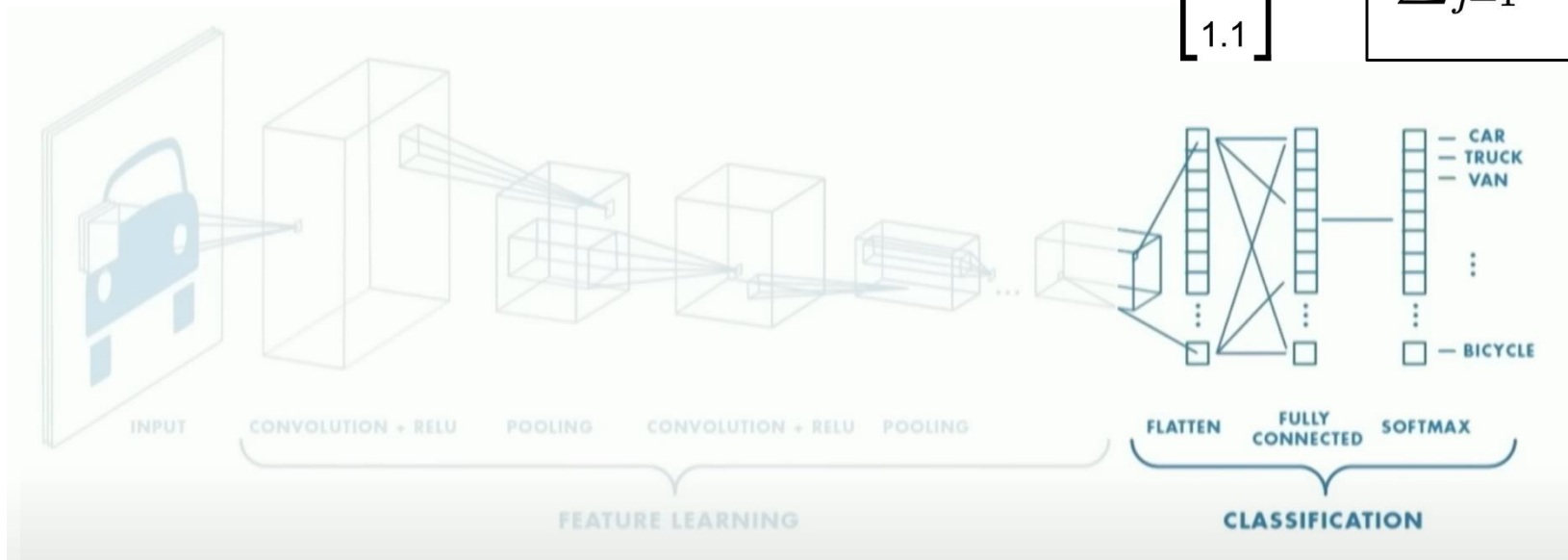
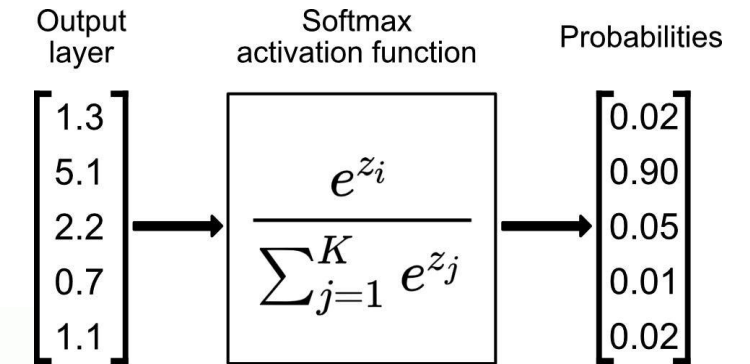
1. Learn features in input images through **convolution**
2. Introduce **non-linearity** through activation function (real-world data is nonlinear!)
3. Reduce dimensionality and preserve spatial invariance with **pooling**



[source: introtodeeplearning.com]

Summary – CNN: Classification

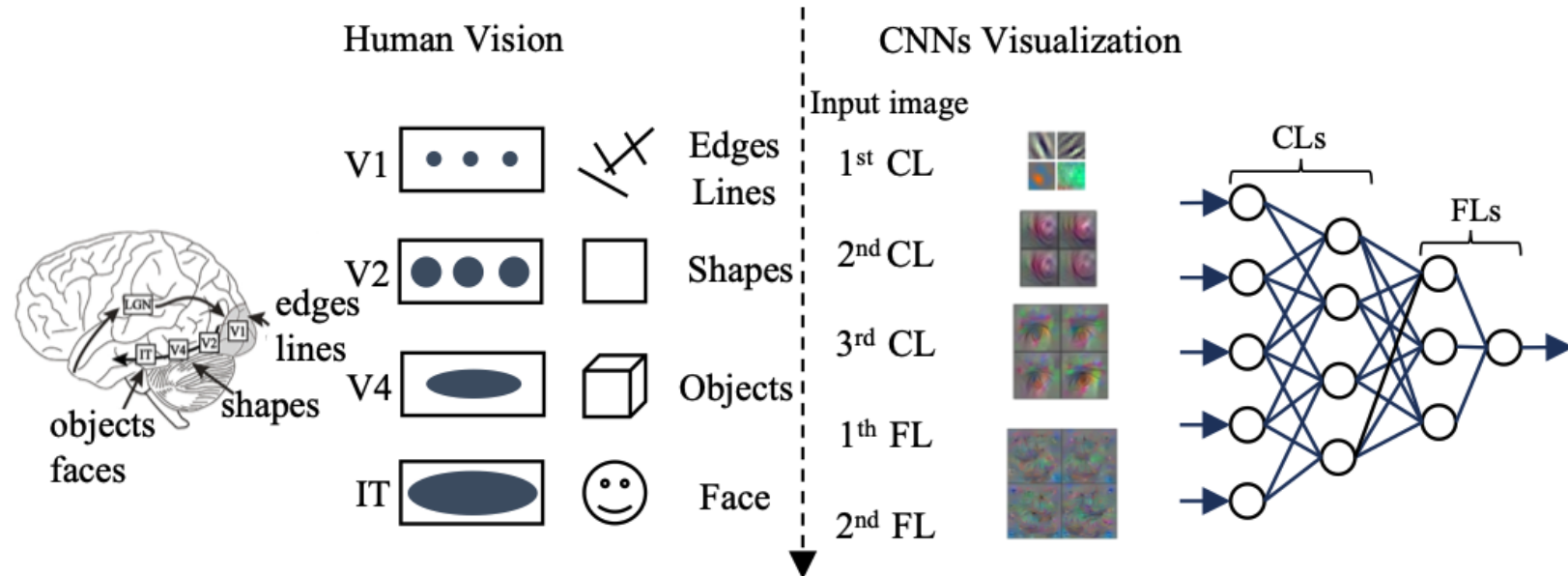
1. CONV and POOL layers output high-level features of input
2. Fully connected layers uses those features for classifying input images
3. Express output as probability of an image to belong to a particular class



[source: introtodeeplearning.com]

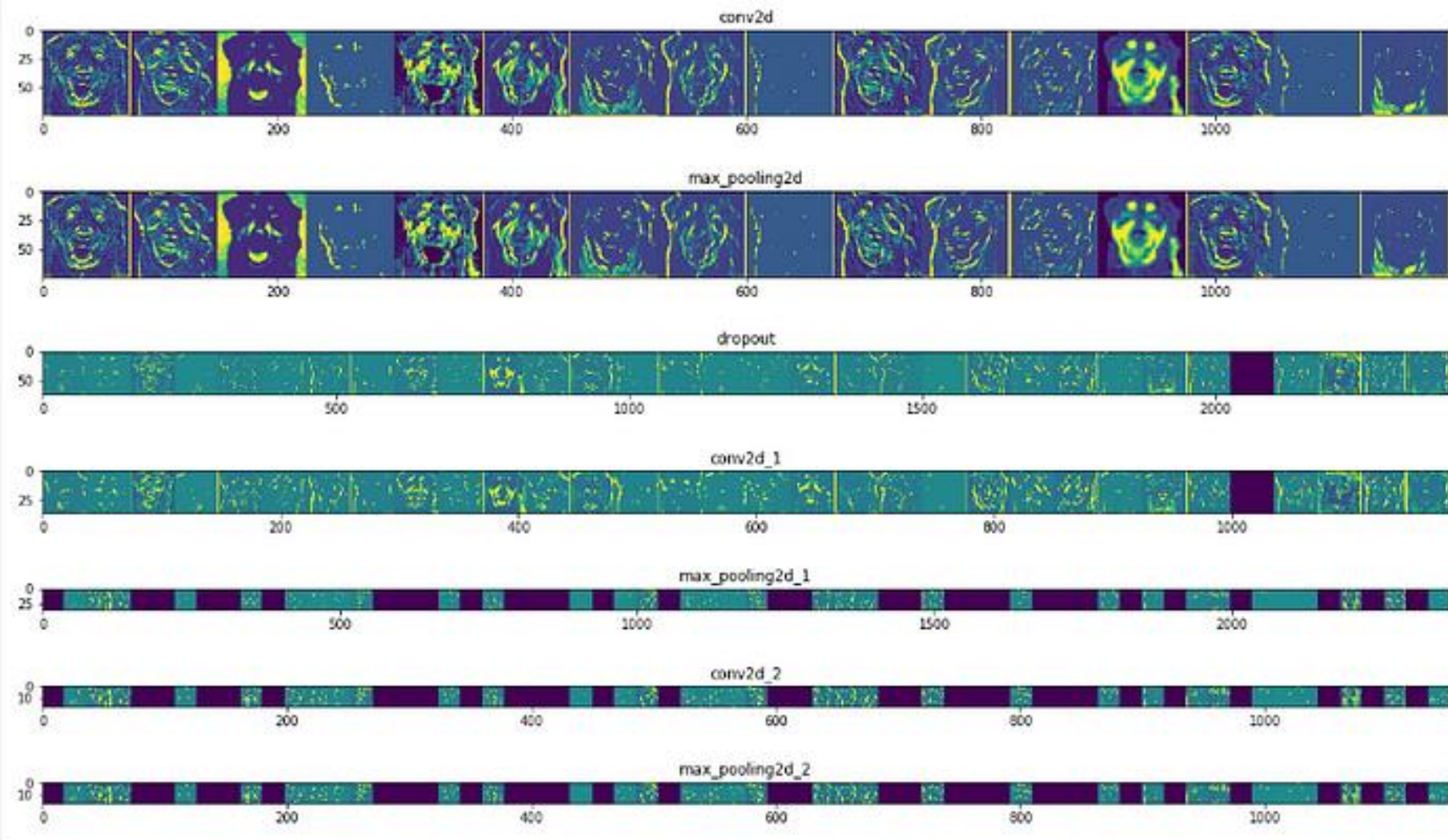
Compare to Human Vision System

- V1, V2, V4: visual neuron areas
 - Receptive fields increases from V1 to V4
 - Process simple visual objects, e.g., curves, edges..
- IT: inferior temporal cortex
 - Process complex visual objects, e.g., a human face.



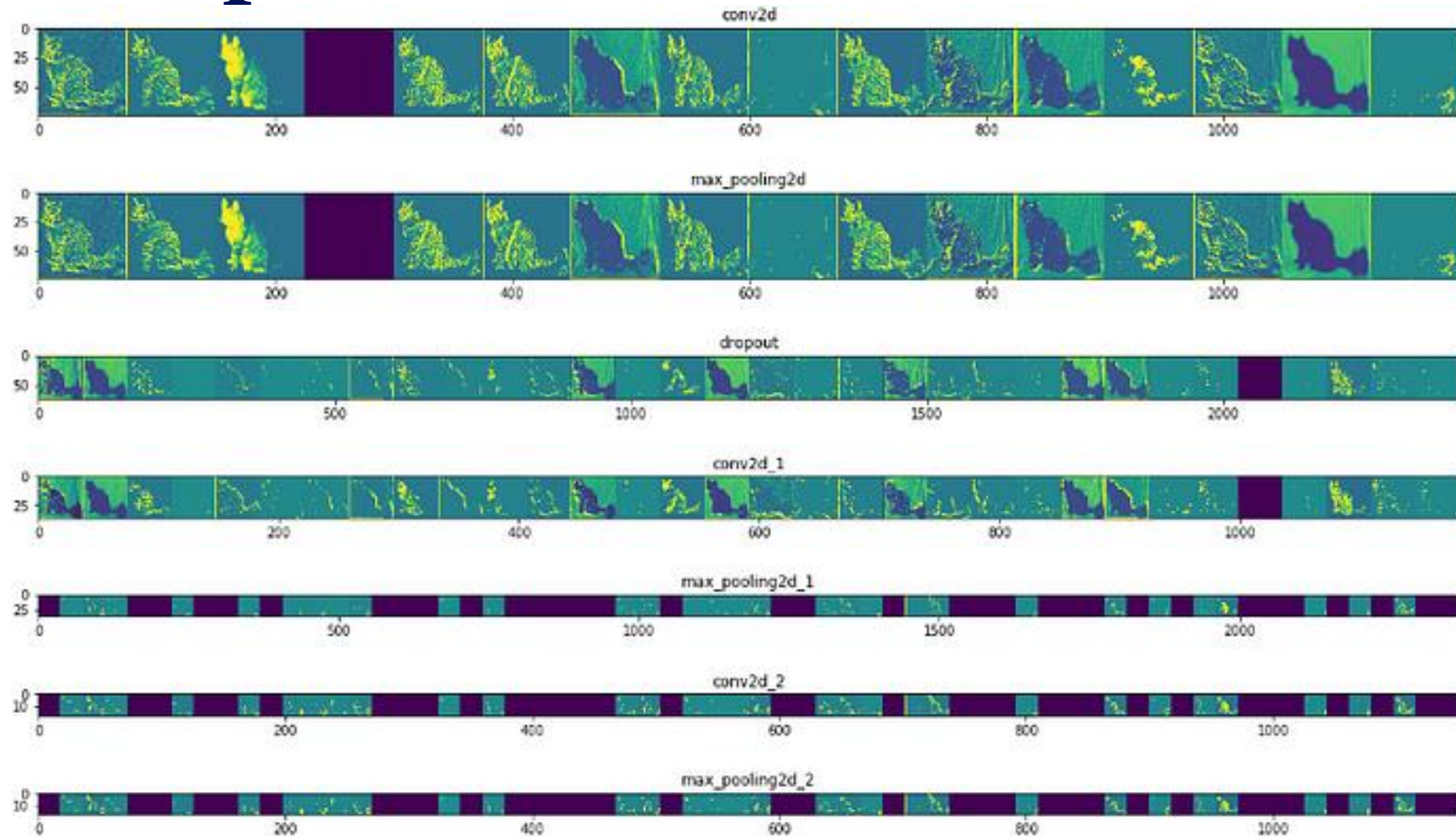
Qin, Zhuwei, Fuxun Yu, Chenchen Liu, and Xiang Chen. "How convolutional neural network see the world-A survey of convolutional neural network visualization methods." *arXiv preprint arXiv:1804.11191* (2018).

Feature Map Visualization



<https://towardsdatascience.com/convolutional-neural-network-feature-map-and-filter-visualization-f75012a5a49c>

Feature Map Visualization



<https://towardsdatascience.com/convolutional-neural-network-feature-map-and-filter-visualization-f75012a5a49c>

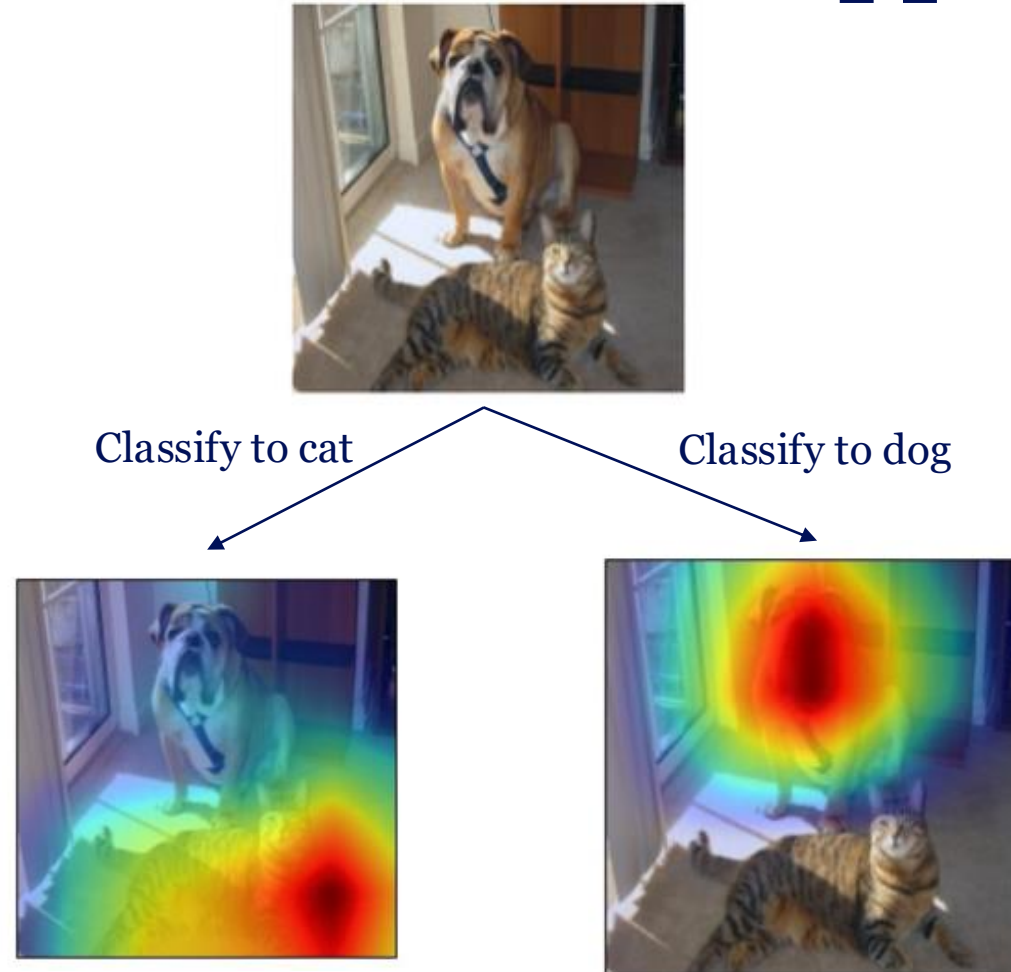
Gradient-weighted Class Activation Mapping

Grad-CAM

- Weighted “overlay” of all feature maps in an CNN
- Each layer is weighted by the magnitude of its gradient

Other visualization methods:

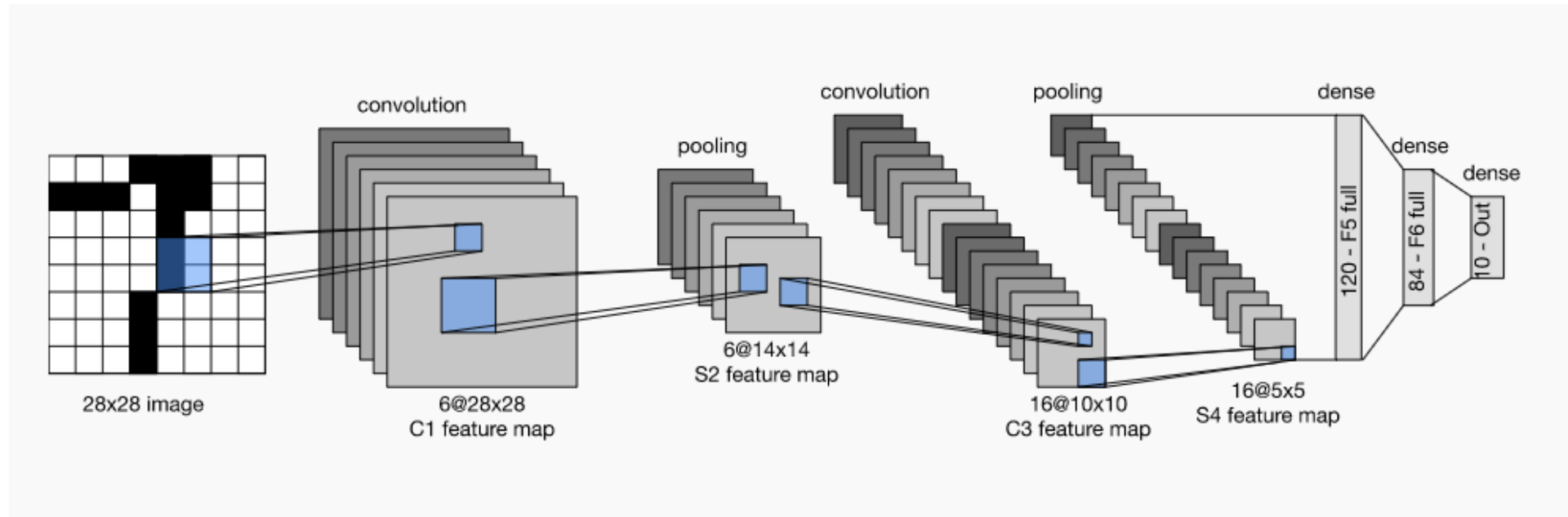
- <https://distill.pub/2017/feature-visualization/>
- <https://arxiv.org/pdf/1804.11191.pdf>



Selvaraju, Ramprasaath R., Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. "Grad-cam: Visual explanations from deep networks via gradient-based localization." In *Proceedings of the IEEE international conference on computer vision*, pp. 618-626. 2017.

Image Classification: LeNet-5

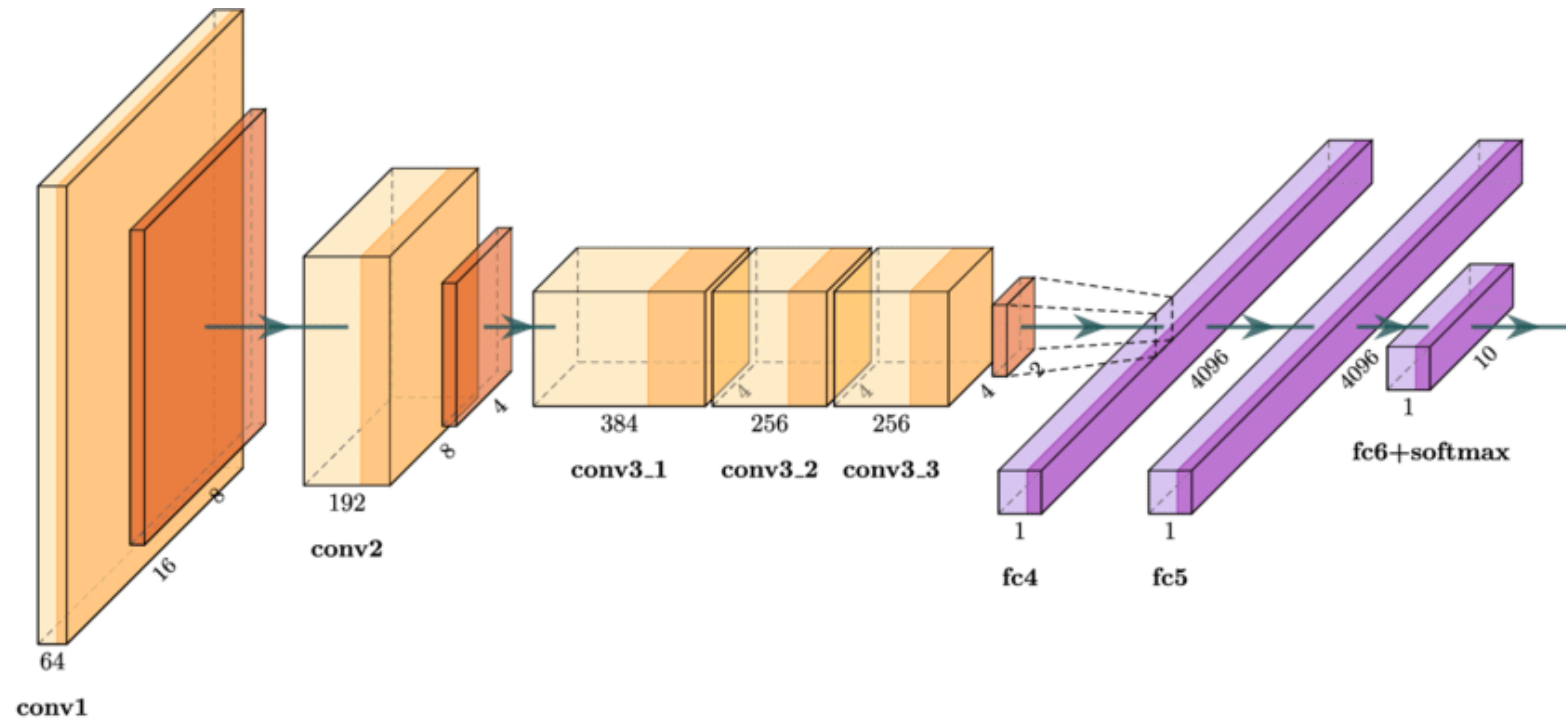
- The first successful deep learning architecture



LeCun, Yann, Bernhard Boser, John S. Denker, Donnie Henderson, Richard E. Howard, Wayne Hubbard, and Lawrence D. Jackel. "Backpropagation applied to handwritten zip code recognition." *Neural computation* 1, no. 4 (1989): 541-551.

Image Classification: AlexNet

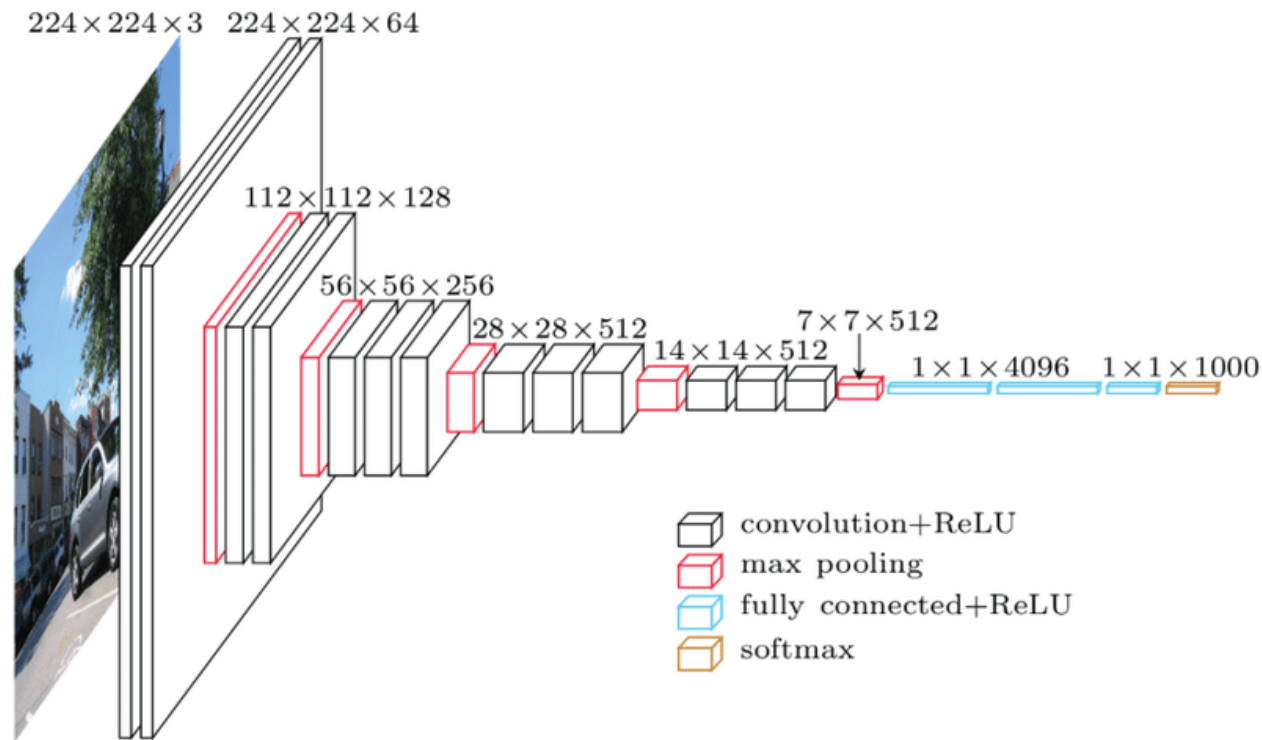
Famous for winning the **ImageNet** Large Scale Visual Recognition



Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." *Advances in neural information processing systems* 25 (2012).

Image Classification: VGG

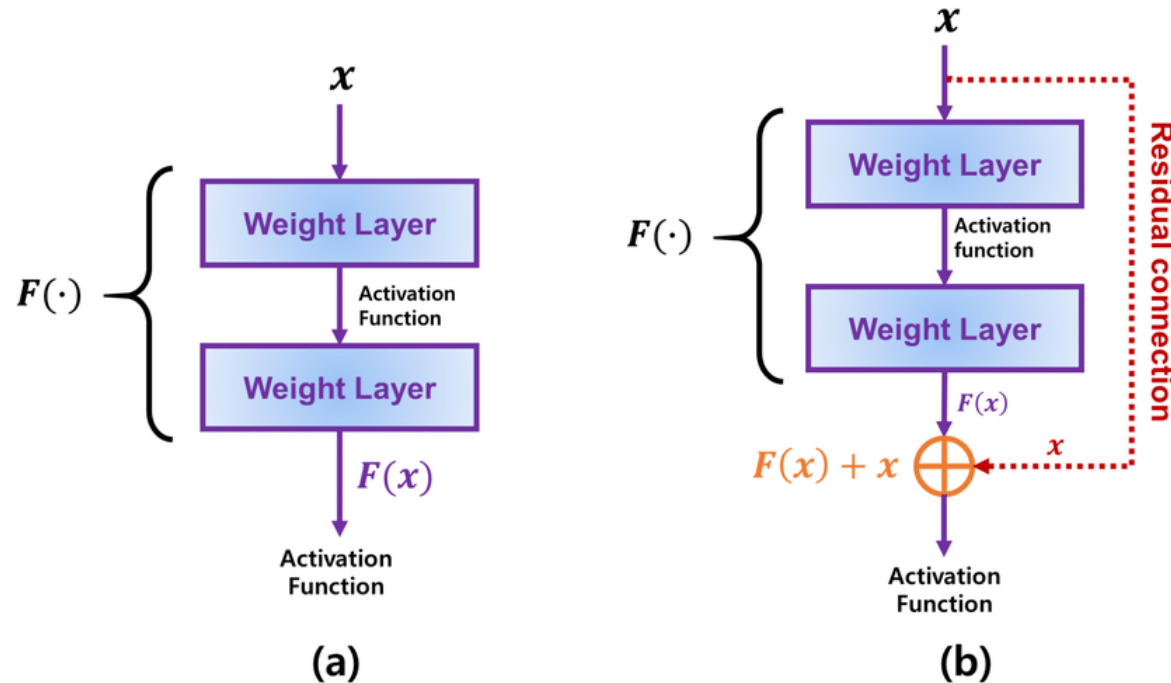
Visual Geometry Group (<https://www.robots.ox.ac.uk/~vgg/>)



Simonyan, Karen, and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition." *arXiv preprint arXiv:1409.1556* (2014).

Wait.. What about vanishing gradient problem?

- More layers -> more expressive power in CNNs
- Gradients at the early convolutional layers will be tiny..
- One solution: **Residual blocks**

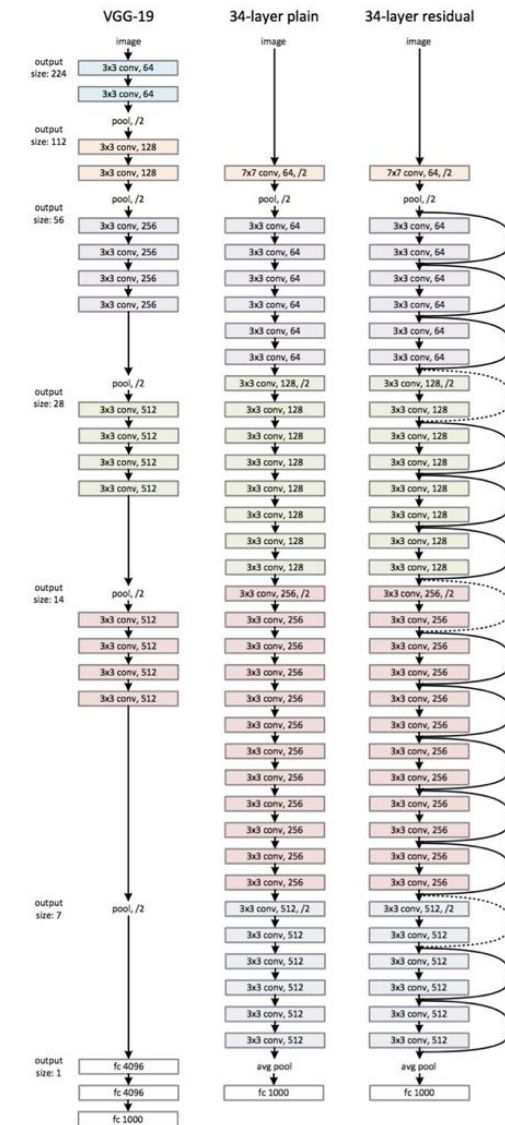


Source: Kim, Kyung-Soo, and Yong-Suk Choi. "HyAdamC: A new adam-based hybrid optimization algorithm for convolution neural networks." *Sensors* 21, no. 12 (2021): 4054.

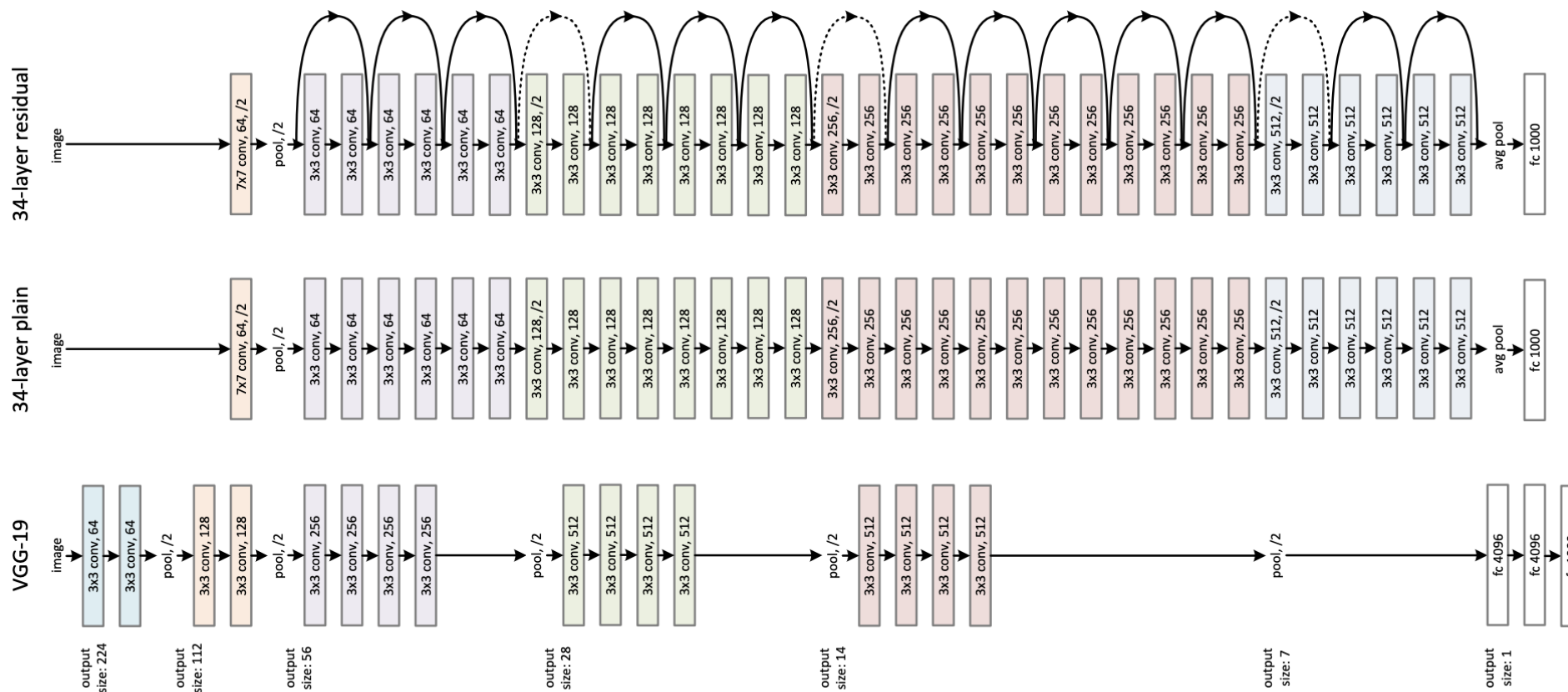
ResNet

- Allows for building up very very deep architectures
- The gradient flow can be backpropagated via the “skip” connections without being down-scaled
- The most used backbone model in many many larger models
 - RCNN (Regions with CNN features) for object detection
 - In Image captioning
 - In Visual Transformer
 - In visual autoencoder
 - ...

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep residual learning for image recognition." In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770-778. 2016.



ResNet



He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep residual learning for image recognition." In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770-778. 2016.



Thank you – Any questions?



Universiteit
Leiden
The Netherlands

e.raponi@liacs.leidenuniv.nl

Discover the world at Leiden University